

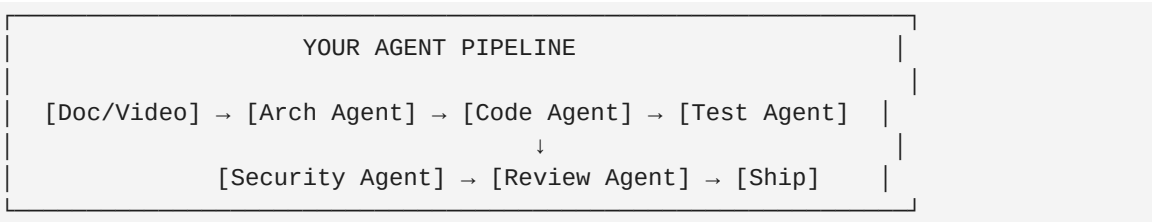
Claude Code Agent Pipeline

Time & Quality Estimates for Order Flow Software Development

Using Specialised Agents for Architecture, Code, Testing & Security

1. The Agent Pipeline Architecture

Using a Claude Code subscription with specialised agents for each phase creates an AI-assisted software factory. Each agent focuses on a single concern, enabling parallel workstreams and consistent quality.



2. Realistic Time Estimates by Phase

Phase 1 — Architecture & Design Agent

Input: Your documents + YouTube videos (trading platform docs, API specs)

Output: Full architecture doc, data flow diagrams, interface contracts

Task	AI Time	Human Review
Ingest docs + extract requirements	10–20 mins	30 mins

Generate architecture document	15–30 mins	1–2 hours
Define C++ class interfaces	20–40 mins	1 hour
API contract (REST + WebSocket)	15–20 mins	1 hour
Phase Total	~1 hour AI	~1 day human

Phase 2 — Code Generation Agent

This is where reality diverges most from expectation. AI generates fast; human integration takes longer.

Module	AI Generation	Human Integration & Fix
Tick ingestion engine (ring buffers)	1–2 hours	1–2 days
Order flow calculations (Delta, Footprint)	2–3 hours	2–3 days
Config system (YAML + hot reload)	30 mins	4 hours
License engine + HW fingerprint	1–2 hours	1–2 days
REST + WebSocket API layer	1–2 hours	1–2 days
NinjaTrader C# adapter	1 hour	1 day
CMake build system	30 mins	2–4 hours
Phase Total	~8–12 hours AI	~2–3 weeks human

Phase 3 — Testing Agent

Task	AI Time	Human Validation
Unit test suite (gtest)	1–2 hours	4–8 hours
Synthetic tick generator	1 hour	4 hours
Historical replay harness	1–2 hours	1 day
API integration tests	1 hour	4–8 hours
Phase Total	~4–6 hours AI	~1 week human

Phase 4 — Security & Hardening Agent

Task	AI Time	Human Validation
Anti-tamper integration	1 hour	2 days
License server backend	2–3 hours	3–5 days
Obfuscation pipeline setup	30 mins	1 day

API auth (JWT + API keys)	1 hour	1–2 days
Phase Total	~4–5 hours AI	~2 weeks human

3. Full Project Timeline

Period	Milestone
Week 1	Architecture docs, interfaces, API contracts
Week 2–4	Core C++ engine generated + integrated
Week 5–6	NinjaTrader adapter + license engine
Week 7–8	Testing suite + historical replay harness
Week 9–10	Security hardening + license server backend
Week 11–12	Beta testing with real tick data on NinjaTrader
Month 4	Sierra Chart adapter
Month 5–6	Public launch v1.0

Speed Comparison

Approach	Estimated Time
Without agents (traditional solo dev)	12–18 months
With Claude Code agent pipeline	3–4 months
Speedup factor	4–5x faster

4. What Limits the Speed — Honest Breakdown

AI is Fast At

- Boilerplate and structural code generation
- Test case generation from specifications
- Documentation and inline comments
- Repeatable patterns: adapters, serializers, config parsers
- Refactoring and reformatting existing code

AI Still Needs You For

- Debugging subtle memory bugs — lock-free queues are notoriously tricky
- Performance profiling on your actual hardware
- NinjaTrader-specific behaviour not in training data
- Business logic decisions — what exactly counts as 'imbalance' in your system
- Live tick data validation — only you can judge if numbers look correct
- Final security certification and obfuscation tool configuration

The Real Bottleneck

The AI agents are not the bottleneck. The actual constraints are:

Bottleneck	Why It Matters
Your decision velocity	How fast you review and approve AI output determines overall pace
Integration testing with real data	AI cannot run your NinjaTrader instance or judge live output
Edge cases in live markets	Only surface with real tick data and real market conditions
Security certification	Themida/VMProtect require manual configuration and testing
Hardware-specific profiling	Latency characteristics vary per machine — must be measured live

5. What Claude Code Specifically Adds

With a Claude Code subscription running coordinated agents:

Feature	Benefit for This Project
Agentic file editing	Agents edit actual files in your repo — not just suggest code
Multi-file awareness	Understands your whole codebase context across all modules
Terminal access	Can run cmake, gtest, and linters directly in your environment
Persistent sessions	Agents remember context across a full working session
Parallel agents	Arch agent and code agent can run simultaneously
Repo-level refactoring	Can restructure entire modules when interfaces change

Realistically with Claude Code running coordinated agents, an experienced developer directing the pipeline could reach v1.0 NinjaTrader release in 8–10 weeks with full security and licensing — versus 9–12 months of traditional solo development.

6. Recommended Agent Execution Order

The sequence of agents matters — interfaces must be approved before code is generated.

Step	Agent	Action	Your Role
1	Arch Agent	Feed entire discussion + NinjaTrader addon docs → generate architecture	Review output
2	Interface Agent	Generate all C++ headers only — no implementation yet	Approve headers (critical 2 hours)
3	—	Header review checkpoint	Most important human review in the project
4	Code Agent	Implement against approved headers	Review + integrate
5	Test Agent	Generate gtest suite in parallel with Code Agent	Run tests, review failures
6	Security Agent	License engine, anti-tamper, obfuscation pipeline	Configure tools manually
7	Review Agent	Final audit of all code for consistency and gaps	Sign off for beta

Why Header Review is the Critical Step

Once interfaces are locked and approved by you, the code and test agents can run in parallel and produce consistent, interlocking output. If interfaces change after code generation starts, both agents must regenerate — wasting significant time. The 2-hour header review investment saves weeks downstream.

End of Document